

Лабораторная работа №8.

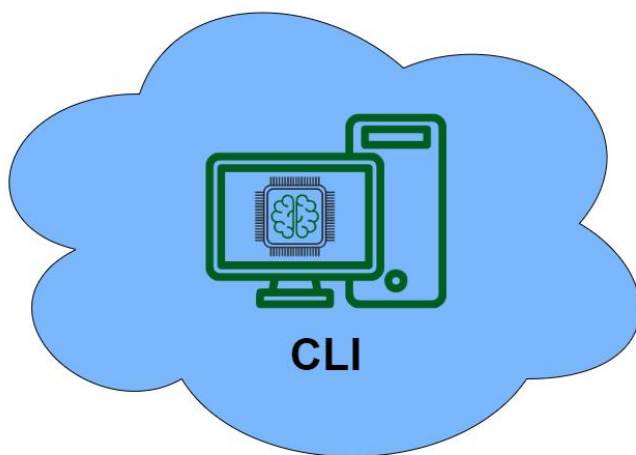
Файлы и их атрибуты

Цель работы:

- Приобрести практический опыт работы с файлами, созданием ссылок. Познакомиться с понятием индексный дескриптор.

Оборудование, ПО:

- ALT1
- Справочная литература или доступ в сеть интернет.



Машина	IP	Маска	Шлюз	DNS
CLI (Альт Рабочая станция 10.4)	- 192.168.1.1	- 24	- -	- 77.88.8.8

Контрольные задания:

1. Исследование inode

Создайте файл и выведите его inode.

Используя команды, изучите все атрибуты файла, связанные с его inode (размер, права доступа, количество жестких ссылок и т.д.).

Удалите файл и создайте новый с тем же именем. Сравните inode нового файла с предыдущим. Объясните результат.

2. Жесткие ссылки

Создайте файл и добавьте к нему две жесткие ссылки.

Проверьте, что все ссылки указывают на один и тот же inode.

Удалите исходный файл и убедитесь, что данные остаются доступными через жесткие ссылки.

Попробуйте создать жесткую ссылку на каталог. Почему это невозможно?

3. Символические ссылки

Создайте файл и символическую ссылку на него с помощью команды `ln -s`.

Удалите исходный файл (а затем создайте заново и переместите в другой каталог). Что происходит при попытке доступа к символической ссылке.

4. Напишите скрипт на Bash, который:

Создает файл и жесткую ссылку на него.

Проверяет, что оба файла имеют одинаковый inode.

Изменяет права доступа на файл и выводит информацию о его атрибутах.

Используемые источники:

- Официальная документация операционной системы Альт Рабочая Станция 10.4 <https://docs.altlinux.org/ru-RU/alt-workstation/10.4/html/alt-workstation/index.html>

Файл — это понятие, привычное любому пользователю компьютера. Для пользователя каждый файл — это отдельный предмет, у которого есть начало и конец и который отличается от всех остальных файлов именем и расположением («как называется» и «где лежит»). Как и любой предмет, файл можно создать, переместить и уничтожить, однако без внешнего вмешательства он будет сохраняться неизменным неопределенно долгое время. Файл предназначен для хранения данных любого типа — текстовых, графических, звуковых, исполняемых программ и многого другого.

Для операционной системы Linux файл — не менее важное понятие, чем для её пользователя: все данные, хранящиеся на любых носителях, обязательно находятся внутри какого-нибудь файла, в противном случае они просто недоступны ни для операционной системы, ни для её пользователей. Более того, все устройства, подключённые к компьютеру (начиная клавиатурой и заканчивая любыми внешними устройствами, например, принтерами и сканерами) Linux представляет как файлы (так называемые файлы-дырки). Конечно, файл, содержащий обычные данные, сильно отличается от файла, предназначенного для обращения к устройству, поэтому в Linux определены несколько различных типов файлов.

В GNU/Linux как и других Unix-подобных операционных системах понятие типа файла не связано с расширением файла (несколькими буквами после точки в конце имени), как это обстоит в Windows.

Unix-подобная ОС не следит за расширениями файлов. Задача связать расширения файла с конкретным пользовательским приложением, в котором этот файл будет открываться, видимо лежит на какой-либо дополнительной программе. В свою очередь пользовательское приложение анализирует структуру данных файла, расширение ему также безразлично.

Таким образом, среди файловых атрибутов, хранящихся в операционной системе на базе ядра Linux, нет информации о типе данных в файле. Там есть информация о более существенном разделении, связанном с тем, что в Unix-подобных системах все объекты — это файлы. Все объекты весьма разнообразны. Поэтому тип файла в Linux — это скорее тип объекта, но не тип данных как в Windows.

В операционной системе GNU/Linux существуют следующие типы файлов: обычные файлы, каталоги, символьные ссылки, блочные устройства, символьные устройства, сокеты, каналы. Каждый тип имеет собственное обозначение одним символом.

Типы файлов		Назначение
Обычные файлы	-	Хранение символьных и двоичных данных
Каталоги	d	Организация доступа к файлам
Символьные ссылки	l	Предоставление доступа к файлам, расположенных на любых носителях
Блочные устройства	b	Предоставление интерфейса для взаимодействия с аппаратным обеспечением компьютера
Символьные устройства	c	
Каналы	p	Организация взаимодействия процессов в операционной системе

Обычные файлы. Это самый распространенный тип файлов. Сюда относятся все файлы с данными, играющими роль ценной информации сами по себе. Это могут быть файлы изображений, сжатые файлы, текстовые файлы, файлы программ и другое. В любом случае это будет обычный (regular) файл. Все они обозначаются знаком минус "-".

Каталоги d. В Linux каталог представляет собой такой тип файла, данными которого является список имен других файлов и каталогов, вложенных в данный каталог. Напрямую, то есть через какой-либо редактор, пользователь не может редактировать данные файла-каталога. Редактированием занимается ядро операционной системы, получая, в том числе от пользователя, команды создания файла, удаления и др.

Символьная ссылка. Это файл, который служит ссылкой на другой файл или каталог. Выполнение символьной ссылки приводит к открытию файла, на который она указывает. Если удалить исходный файл, то символьная ссылка продолжит существовать. Она по-прежнему будет указывать на файл, которого уже нет.

Символьные и блочные устройства. Устройства в Linux тоже представлены файлами, для них даже выделен каталог /dev который хранит виртуальную файловую систему devfs. Эта файловая система хранит список всех устройств компьютера. Файлы устройств предназначены для обращения к аппаратному обеспечению компьютера (дискам, принтерам, терминалам и др.). Когда происходит обращение к файлу устройства, то ядро операционной системы передает запрос драйверу этого

устройства.

Блочные устройства — это диски и их разделы, raid-массивы, и тому подобное. Эти устройства могут хранить файловую систему и файлы на ней. Они умеют обрабатывать операции ввода-вывода, то есть умеют записывать или считывать блоки данных. И обычно поддерживают произвольный доступ к данным.

Символьные устройства — это COM-порты, LPT-порты, PS/2-мышки и клавиатуры, USB-мышки и клавиатуры. Такие устройства обычно поддерживают операции (read, write, open, close). И поддерживают посимвольный, то есть последовательный доступ к данным.

Утилита file

Утилита file используется для определения типа файла.

Она анализирует содержимое файла и пытается определить его формат.

```
$ file [параметры] файл1 ...
```

Для простого исследования типов файлов достаточно передать имена этих файлов утилите без использования каких-либо параметров.

```
user@host-15 ~ $ file 1.txt
1.txt: ASCII text
user@host-15 ~ $ file new-year.jpg
new-year.jpg: JPEG image data, JFIF standard 1.01
user@host-15 ~ $ file new-year-photo.tar.gz
new-year-photo.tar.gz: gzip compressed data, from Unix
```

Утилита file может обрабатывать множества файлов. Для этого достаточно использовать шаблоны командной оболочки.

```
user@host-15 ~ $ file new-year*
new-year: JPEG image data, JFIF standard 1.01
new-year.jpg: JPEG image data, JFIF standard 1.01
new-year-photo.tar.gz: gzip compressed data, from Unix
user@host-15 ~ $
```

Индексный дескриптор (inode) — это структура данных, содержащая метаданные файла, а также указатели на блоки с данными файла. В файловой системе Linux, например, Ext4, у файла есть не только само его содержимое, например, тот текст, но и метаданные, такие как дата создания, доступа, модификации и права. Вот эти метаданные и хранятся в Inode. У каждого файла есть своя уникальная Inode и именно здесь указано в каких блоках находятся данные файла.

Индексный дескриптор хранит следующую информацию:

- свой номер;
 - тип файла;
 - владельца файла и права доступа к нему;
 - время:
 - создания файла (**crtime**);
 - доступа к файлу, его ещё называют временем касания (**atime**);
 - последнего изменения файла (**mtime**);
 - последнего изменения метаданных файла (например, изменили права доступа к файлу) (**ctime**);
- физическое расположение блоков данных на диске.

Блоки с данными файла – сегменты на диске, где фактически хранятся данные файла. Inode содержит указатели на эти блоки.

Чтобы увидеть номера индексных дескрипторов к команде `ls` добавляют опцию `-li`. В выводе возле каждого файла написан номер его индексного дескриптора.

Ссылки на inode

Каждый файл в файловой системе имеет по крайней мере одну ссылку. Это имя файла в каталоге, которое указывает на inode.

Жесткие ссылки – дополнительная запись в каталоге, которая указывает на тот же inode, что и оригинальный файл. Файл будет считаться удаленным только тогда, когда количество ссылок на его inode достигнет нуля.

Чтобы посмотреть некоторые метаданные файла, можете воспользоваться командой **stat**.

Пример вывода:

- **File:** `/etc/ssh/ssh_config` — путь к файлу и его имя;
- **Size:** `3281` — размер файла в байтах;
- **regular file** — тип файла (обычный файл);
- **Inode:** `394570` — номер индексного дескриптора;
- **Links:** `1` — количество жестких ссылок (про жёсткие и мягкие ссылки будет написана следующая статья);
- **Access:** `(0644/-rw-r—r—)` **Uid:** `( 0/ root)` **Gid:** `( 0/ root)` —

владелец и права доступа (это тоже разберём в других статьях);

- **Access:** — время последнего доступа к файлу;
- **Modify:** — время последнего изменения файла;
- **Change:** — время последнего изменения метаданных файла;
- **Birth:** — время создания файла

Структура inode

Структура inode (index node) содержит всю информацию о файле, кроме его имени. Имя файла хранится в каталоге, который ссылается на inode.

Удаление файла происходит путем удаления его записи в каталоге и уменьшения счетчика ссылок на inode. Если счетчик ссылок достигает нуля, система освобождает inode и блоки данных для использования другими файлами

Жесткие ссылки

Жесткие ссылки позволяют иметь несколько имен для одного файла в разных местах файловой системы без дублирования самого файла. Они указывают на тот же inode и, следовательно, на те же данные на диске.

Создание жесткой ссылки:

```
$ ln first second  
$ ln /home/user1/file1 /home/user2/file1
```

Особенности жестких ссылок:

- **Равноправие ссылок**

Все жесткие ссылки на файл равноправны; система не различает "оригинальное" имя файла и жесткие ссылки на него.

- **Удаление ссылок**

Удаление одной из жестких ссылок не влияет на файл или другие ссылки, до тех пор, пока хотя бы одна ссылка остается.

- **Перемещение жесткой ссылки**

Жесткую ссылку можно переместить в пределах той же файловой системы без влияния на файл.

- **Копирование жесткой ссылки**

Создание новой жесткой ссылки не создает копию данных файла, она просто добавляет еще одну запись в каталог, указывающую на тот же inode.

Ограничения

Жесткие ссылки нельзя создавать на каталоги (для предотвращения возможности создания циклических ссылок) и не могут пересекать файловые системы.

Утилита ln используется для создания ссылок. Чтобы использовать команду ln, откройте окно терминала и введите команду в следующем формате:

```
ln [-sf] [source] [destination]
```

- По умолчанию команда ln создает hard link (жесткая ссылка).
- Используйте параметр -s, чтобы создать символическую ссылку, она же soft link.
- Параметр -f заставит команду перезаписать уже существующий файл.
- **Source** - это файл или каталог, на который делается ссылка.
- **Destination** - это место для сохранения ссылки - если это поле не заполнено, символическая ссылка сохраняется в текущем рабочем каталоге.

Например, создайте символическую ссылку с помощью:

```
$ ln -s test_file.txt link_file.txt
```

Создает символическую ссылку link file.text, которая указывает на testfile.txt.

Чтобы проверить, создана ли символическая ссылка, используйте команду ls:

```
$ ls -l link_file.txt
```

Мягкие ссылки

Символические или мягкие ссылки представляют собой специальные файлы в файловой системе, которые вместо пользовательских данных содержат путь к файлу, на который они указывают. Это позволяет иметь несколько имен для одного файла или каталога, а также облегчает организацию структуры файлов на компьютере. Символические ссылки работают как гиперссылки в Интернете.

Синтаксис создания мягкой ссылки:

```
$ ln -s ../user1/file1 link1
```

Особенности мягких ссылок:

- **Копирование и перемещение ссылки**

Символические ссылки могут быть скопированы или перемещены как обычные файлы.

- **Перемещение целевого файла**

Если целевой файл перемещен или удален, символическая ссылка будет указывать в "никуда", т.е. работать не будет.

- **Ограничения**

Символическая ссылка может указывать только на файл или каталог внутри той же файловой системы, в которой она создана.

- **Быстрый доступ**

Символические ссылки часто используются для быстрого доступа к часто используемым файлам без ввода всего местоположения.

Если исходный файл будет перемещен, удален или станет недоступным (например, сервер отключится), ссылку нельзя будет использовать. Чтобы удалить символическую ссылку, используйте команду `rm (remove)` или `unlink`.

```
$ rm link_file.txt
```

```
$ unlink link_file.txt
```

Сравнение мягких и жестких ссылок:

- **По области действия:**

Жесткие ссылки действуют только в пределах одной файловой системы, тогда как символические ссылки могут указывать на файлы и каталоги в других файловых системах.

- **По действию удаления файла:**

Жесткая ссылка на файл остается функциональной до тех пор, пока существует хотя бы одна ссылка. Символическая ссылка становится недействительной, если оригинальный файл удаляется.